

## 1. ZADATAK

## PRII – G2

Izvršiti definiciju funkcija na način koji odgovara opisu (komentarima) datom neposredno uz pozive ili nazive funkcija. Možete dati komentar na bilo koju liniju code-a koju smatrate da bi trebalo unaprijediti ili da će eventualno uzrokovati grešku prilikom kompajliranja. Također, možete dodati dodatne funkcije ili metode koje će vam olakšati implementaciju programa.

```
#include <iostream>
using namespace std;

string crt = "\n-----\n";

string PORUKA_TELEFON = crt +
"TELEFONE ISKLJUCITE I ODLOZITE U TORBU, DZEP ILI DRUGU LOKACIJU VAN
DOHVATA.\n"
"CESTO SE NA TELEFONIMA (PRO)NALAZE PROGRAMSKI KODOVI KOJI MOGU BITI
ISKORISTENI ZA\n"
"RJESAVANJE ISPITNOG ZADATKA, STO CE, U SLUCAJU PRONALASKA, BITI
SANKCIONISANO.\n\n"
"SVI KOD KOJIH BUDE PRONADJEN TELEFON ILI NEKI DRUGI UREDJAJ KOJI MOZE
SADRZAVATI\n"
"MATERIJAL ILI RJESENJA ISPITNIH ZADATAKA CE BITI UDALJENI SA ISPITA I
BIT CE POKRENUT\n"
"POSTUPAK PROTIV NJIH" + crt;

string PORUKA_ISPIT = crt +
"0. PROVJERITE DA LI ZADACI PRIPADAJU VASOJ GRUPI (G1/G2)\n"
"1. SVE KLASSE SA DINAMICKOM ALOKACIJOM MORAJU IMATI ISPRAVAN
DESTRUKTOR\n"
"2. IZOSTAVLJANJE DESTRUKTORA ILI NJEGOVIH DIJELOVA BIT CE OZNACENO
KAO TM\n"
"3. ATRIBUTI, METODE I PARAMETRI MORAJU BITI IDENTICNI ONIMA U TESTNOJ
MAIN FUNKCIJI,\n"
"    OSIM AKO POSTOJI JASNO OPISAN RAZLOG ZA MODIFIKACIJU\n"
"4. IZUZETKE BACAJTE SAMO TAMO GDJE JE IZRICITO NAGLASENO\n"
"5. SVE METODE KOJE SE POZIVAJU U MAIN-U MORAJU POSTOJATI.\n"
"    AKO NEMATE ZELJENU IMPLEMENTACIJU, OSTAVITE PRAZNO TIJELO ILI
VRATITE DEFAULT VRIJEDNOST\n"
"6. U MAIN FUNKCIJI MOZETE DODAVATI TESTNE PODATKE I POZIVE PO
VLASTITOM IZBORU\n"
"7. TESTIRAJTE PROGRAM U OBA MODA (F5 i Ctrl+F5)" + crt;

char* AlocirajTekst(const char* tekst) {
    if (tekst == nullptr) return nullptr;
    size_t velicina = strlen(tekst) + 1;
    char* noviTekst = new char[velicina];
    strcpy_s(noviTekst, velicina, tekst);
    return noviTekst;
}

enum StatusZahtjeva { ZAPRIMLJEN, DIJAGNOSTIKA, POPRAVKA, ZAVRSEN };
const char* StatusZahtjevaNazivi[] = {
    "ZAPRIMLJEN", "DIJAGNOSTIKA", "POPRAVKA", "ZAVRSEN"
};

template<class T1, class T2, int max>
```

```

class Kolekcija {
    T1* _elementi1;
    T2* _elementi2;
    int _trenutno;
public:
    Kolekcija() : _trenutno(0) {
        _elementi1 = new T1[max];
        _elementi2 = new T2[max];
    }
    int GetTrenutno() const { return _trenutno; }
    const T1& GetElement1(int indeks) const { return
_elementi1[indeks]; }
    const T2& GetElement2(int indeks) const { return
_elementi2[indeks]; }

    friend ostream& operator<<(ostream& COUT, const Kolekcija& obj) {
        for (int i = 0; i < obj.GetTrenutno(); i++)
            COUT << obj.GetElement1(i) << " " << obj.GetElement2(i) <<
endl;
        return COUT;
    }
    ~Kolekcija() {
        delete[] _elementi1;
        delete[] _elementi2;
        _elementi1 = nullptr;
        _elementi2 = nullptr;
        _trenutno = 0;
    }
};

class DatumVrijeme {
    int* _dan, * _mjesec, * _godina, * _sati, * _minute;
public:
    DatumVrijeme(int dan = 1, int mjesec = 1, int godina = 2000,
        int sati = 0, int minute = 0) {
        _dan = new int(dan);
        _mjesec = new int(mjesec);
        _godina = new int(godina);
        _sati = new int(sati);
        _minute = new int(minute);
    }
    int GetGodina() const { return *_godina; }
    friend ostream& operator<<(ostream& COUT, const DatumVrijeme& obj)
{
    // ToString vraca datum i vrijeme u formatu DD.MM.GGGG HH:MM
    COUT << obj.ToString();
    return COUT;
}
    ~DatumVrijeme() {
        delete _dan;
        delete _mjesec;
        delete _godina;
        delete _sati;
        delete _minute;
        _dan = _mjesec = _godina = _sati = _minute = nullptr;
    }
};

```

```

class Intervencija {
    string _opis;
    string _tehtnicar;
    double _cijena;
    int _trajanjeMinuta;
public:
    const string& GetOpis() const { return _opis; }
    const string& GetTehnicar() const { return _tehtnicar; }
    double GetCijena() const { return _cijena; }
    int GetTrajanjeMinuta() const { return _trajanjeMinuta; }
    friend ostream& operator<<(ostream& COUT, const Intervencija& obj)
{
    /*
    ToString vraca podatke u formatu :
    opis | tehnicar | cijena KM | trajanje min
    Zamjena diska | Haris Hadzic | 85.50 KM | 45 min
    */
    COUT << obj.ToString();
    return COUT;
}
};

class Klijent {
    string _imePrezime;
    string _email;
    string _telefon;
public:
    const string& GetImePrezime() const { return _imePrezime; }
    const string& GetEmail() const { return _email; }
    const string& GetTelefon() const { return _telefon; }
    friend ostream& operator<<(ostream& COUT, const Klijent& obj) {
        COUT << obj.GetImePrezime() << " | " << obj.GetEmail()
            << " | " << obj.GetTelefon();
        return COUT;
    }
};

class ServisniZahtjev {
    char* _oznaka;
    char* _uredjaj;
    char* _opisKvara;
    int _redniBroj;
    Klijent _klijent;
    Kolekcija<StatusZahtjeva, DatumVrijeme, 10> _statusi;
    vector<Intervencija> _intervencije;
public:
    const char* GetOznaka() const { return _oznaka; }
    const char* GetUredjaj() const { return _uredjaj; }
    const char* GetOpisKvara() const { return _opisKvara; }
    int GetRedniBroj() const { return _redniBroj; }
    Klijent& GetKlijent() { return _klijent; }
    const Klijent& GetKlijent() const { return _klijent; }
    Kolekcija<StatusZahtjeva, DatumVrijeme, 10>& GetStatusi() {
return _statusi; }
    const Kolekcija<StatusZahtjeva, DatumVrijeme, 10>& GetStatusi()
const { return _statusi; }
    vector<Intervencija>& GetIntervencije() { return _intervencije; }

```

```

    const vector<Intervencija>& GetIntervencije() const { return
_intervencije; }
    StatusZahtjeva GetTrenutniStatus() const { return
_statusi.GetElement1(_statusi.GetTrenutno() - 1); }
    friend ostream& operator<<(ostream& COUT, const ServisniZahtjev&
obj) {
        // ToString vraca:
        // oznaka | ime i prezime klijenta | uredjaj | trenutni status
        COUT << obj.ToString();
        return COUT;
    }
    ~ServisniZahtjev() {
        delete[] _oznaka;
        delete[] _uredjaj;
        delete[] _opisKvara;
        _oznaka = _uredjaj = _opisKvara = nullptr;
    }
};

class Servis {
    char* _naziv;
    vector<ServisniZahtjev> _zahtjevi;
public:
    Servis(const char* naziv = "") {
        _naziv = AlocirajTekst(naziv);
    }
    const char* GetNaziv() const { return _naziv; }
    vector<ServisniZahtjev>& GetZahtjevi() { return _zahtjevi; }
    const vector<ServisniZahtjev>& GetZahtjevi() const { return
_zahhtjevi; }
    ~Servis() {
        delete[] _naziv; _naziv = nullptr;
    }
};

const char* GetOdgovorNaPrvoPitanje() {
    cout << "Pitanje -> Pojasnite zbog cega se operator<< implementira
kao globalna a ne funkcija clanica, te na koji nacin se implementiraju
prefiksadni i postfiksadni oblik operatora inkrementiranja?\n";
    return "Odgovor -> OVDJE UNESITE VAS ODGOVOR";
}

const char* GetOdgovorNaDrugoPitanje() {
    cout << "Pitanje -> Pojasnite nacin na koji biste, koristeci
obradjene klase i metode, mogli saznati velicinu nekog tekstualnog
fajla.\n";
    return "Odgovor -> OVDJE UNESITE VAS ODGOVOR";
}

int main() {
    cout << PORUKA_TELEFON; cin.get();
    cout << PORUKA_TELEFON; cin.get();
    cout << PORUKA_ISPIT; cin.get();
    system("cls");

    cout << GetOdgovorNaPrvoPitanje() << crt;
    cin.get();
    cout << GetOdgovorNaDrugoPitanje() << crt;
    cin.get();

```

```
/*
Funkcija GenerisiOznaku generise oznaku servisnog zahtjeva u
formatu:
SRV-BBB/IN-GGGG

Potpis funkcije treba biti:
string GenerisiOznaku(const char* imePrezime, int redniBroj, int
godina)

SRV -> fiksni prefiks,
BBB -> redni broj zahtjeva popunjen nulama na slobodnim mjestima,
IN -> inicijali imena i prezimena klijenta,
GGGG -> godina zaprimanja zahtjeva.

Kod imena koje sadrzi vise rijeci koriste se inicijal prve i
posljednje
rijeci. Redni broj mora biti u rasponu 1-999, a godina 2000-2099.
Za neispravne podatke funkcija vraca "SRV-000/XX-0000".
*/
if (GenerisiOznaku("Amina Buric", 42, 2026) == "SRV-042/AB-2026")
    cout << "Oznaka OK" << crt;
if (GenerisiOznaku("Goran Skondric", 7, 2026) == "SRV-007/GS-
2026")
    cout << "Oznaka OK" << crt;
if (GenerisiOznaku("Ana Marija Kovac", 156, 2027) ==
"SRV-156/AK-2027")
    cout << "Oznaka OK" << crt;
if (GenerisiOznaku("Amina", 42, 2026) == "SRV-000/XX-0000" &&
    GenerisiOznaku("Amina Buric", 0, 2026) == "SRV-000/XX-0000" &&
    GenerisiOznaku("Amina Buric", 1000, 2026) == "SRV-000/XX-0000"
&&
    GenerisiOznaku("Amina Buric", 42, 1999) == "SRV-000/XX-0000")
    cout << "Neispravni podaci za oznaku OK" << crt;

/*
Koristeci regex, funkcija ValidirajOznaku provjerava prethodno
definisani
format. Prefiks mora biti SRV, redni broj mora imati tri cifre i
ne moze
biti 000, inicijali moraju biti velika slova, a godina je 2000-
2099.

Potpis funkcije treba biti:
bool ValidirajOznaku(const string& oznaka)
*/
if (ValidirajOznaku("SRV-042/AB-2026"))
    cout << "OZNAKA VALIDNA" << crt;
if (!ValidirajOznaku("SRV/042-AB-2026") &&
    !ValidirajOznaku("SRV-42/AB-2026") &&
    !ValidirajOznaku("SRV-042/Ab-2026") &&
    !ValidirajOznaku("SRV-000/AB-2026") &&
    !ValidirajOznaku("2026-SRV-042/AB"))
    cout << "OZNAKA NIJE VALIDNA" << crt;

Kolekcija<int, string, 6> brojevi;
brojevi.Dodaj(10, "Deset");
brojevi.Dodaj(20, "Dvadeset");
```

```
brojevi.Dodaj(10, "Deset");
brojevi.Dodaj(30, "Trideset");
brojevi.Dodaj(20, "Dvadeset");
cout << brojevi << crt;

/*
UkloniDuplikate vraca novu kolekciju u kojoj je zadrzano samo prvo
pojavljivanje svakog para. Par predstavlja duplikat iskljucivo ako
su
jednaki i prvi i drugi element. Originalna kolekcija ostaje
nepromijenjena.
*/
Kolekcija<int, string, 6> bezDuplikata =
brojevi.UkloniDuplikate();
cout << "Bez duplikata:" << crt << bezDuplikata;
cout << "Original:" << crt << brojevi;

try {
    Kolekcija<int, string, 2> puna;
    puna.Dodaj(1, "Jedan");
    puna.Dodaj(2, "Dva");
    puna.Dodaj(3, "Tri");
}
catch (exception& e) {
    cout << "Exception: " << e.what() << crt;
}

Kolekcija<int, string, 6> kopijaBrojeva = brojevi;
kopijaBrojeva[0] = 100;
Kolekcija<int, string, 6> dodijeljeniBrojevi;
dodijeljeniBrojevi = brojevi;
dodijeljeniBrojevi.GetElement2(0) = "Izmijenjeno";
cout << "Original:" << crt << brojevi;
cout << "Kopija:" << crt << kopijaBrojeva;
cout << "Dodijeljeni objekat:" << crt << dodijeljeniBrojevi;

DatumVrijeme zaprimljen(9, 7, 2026, 8, 0);
DatumVrijeme dijagnostika(9, 7, 2026, 9, 0);
DatumVrijeme popravka(9, 7, 2026, 10, 0);
DatumVrijeme završen(9, 7, 2026, 11, 0);

/*
ToString vraca datum i vrijeme u formatu DD.MM.GGGG HH:MM,
ukljucujuci
pocetne nule.
*/
cout << zaprimljen.ToString() << crt; // 09.07.2026 08:00
if (dijagnostika > zaprimljen)
    cout << "Vrijeme dijagnostike je nakon zaprimanja" << crt;
DatumVrijeme kopijaDatuma(dijagnostika);
if (kopijaDatuma == dijagnostika && !(zaprimljen == dijagnostika))
    cout << "Provjera vremena, OK." << crt;

Klijent amina("Amina Buric", "amina@fit.ba", "061-111-222");
Klijent goran("Goran Skondric", "goran@fit.ba", "062-222-333");
Klijent kopijaKlijenta = amina;
cout << kopijaKlijenta << crt;
```

```
/*
    Intervencija sadrzi opis, ime tehnicara, cijenu i trajanje u
    minutama.
*/
    Intervencija pregled("Dijagnostika uredjaja", "Haris Hadzic", 20,
30);
    Intervencija zamjenaDiska("Zamjena diska", "Haris Hadzic", 85.5,
45);
    Intervencija instalacija("Instalacija sistema", "Maja Majic", 35,
60);
/* ToString vraca podatke u formatu:
    opis | tehnicar | cijena KM | trajanje min
    Zamjena diska | Haris Hadzic | 85.50 KM | 45 min
*/
    cout << zamjenaDiska.ToString() << crt;

/*
    Konstruktor ServisnogZahtjeva generise oznaku na osnovu klijenta,
    rednog broja i godine zaprimanja, te evidentira pocetni status
    ZAPRIMLJEN
    sa prosljedjenim vremenom.
*/
    ServisniZahtjev laptop("Laptop", "Ne pokrece se", amina, 42,
    zaprimljen);
    ServisniZahtjev telefon("Telefon", "Razbijen ekran", goran, 7,
    zaprimljen);

/*
    ToString vraca podatke u formatu:
    oznaka | ime i prezime klijenta | uredjaj | trenutni status
    SRV-042/AB-2026 | Amina Buric | Laptop | ZAPRIMLJEN
*/
    cout << laptop.ToString() << crt;
    if (laptop.ToString() ==
        "SRV-042/AB-2026 | Amina Buric | Laptop | ZAPRIMLJEN")
        cout << "ServisniZahtjev ToString OK" << crt;

/*
    DodajStatus dodaje status samo ako je vrijeme vece od vremena
    posljednjeg
    statusa i ako status predstavlja neposredno sljedecu fazu.

    Dozvoljen slijed je: ZAPRIMLJEN -> DIJAGNOSTIKA -> POPRAVKA ->
    ZAVRSEN

    Preskakanje ili ponavljanje statusa nije dozvoljeno. Nakon statusa
    ZAVRSEN nisu dozvoljene nove promjene. Neuspjesan pokusaj vraca
    false
    bez izmjene kolekcije statusa.
*/
    if (!laptop.DodajStatus(POPRAVKA, dijagnostika))
        cout << "Preskakanje statusa nije dozvoljeno" << crt;
    if (laptop.DodajStatus(DIJAGNOSTIKA, dijagnostika))
        cout << "Status DIJAGNOSTIKA dodan" << crt;
    if (!laptop.DodajStatus(DIJAGNOSTIKA, popravka))
        cout << "Ponavljanje statusa nije dozvoljeno" << crt;

/*
```

```
DodajIntervenciju dodaje intervenciju samo dok je zahtjev u
statusu
DIJAGNOSTIKA ili POPRAVKA, uz cijenu i trajanje vece od nule.
Metoda vraca true ako je intervencija dodana, u suprotnom false.

UkupnaCijena vraca zbir cijena svih intervencija, a UkupnoTrajanje
zbir
njihovih trajanja u minutama.
*/
if (laptop.DodajIntervenciju(pregled))
    cout << "Intervencija dodana" << crt;
if (laptop.DodajStatus(POPRAVKA, popravka))
    cout << "Status POPRAVKA dodan" << crt;
laptop.DodajIntervenciju(zamjenaDiska);
laptop.DodajIntervenciju(instalacija);
cout << "Ukupna cijena: " << laptop.UkupnaCijena() << " KM" <<
crt;
cout << "Ukupno trajanje: " << laptop.UkupnoTrajanje() << " min"
<< crt;

ServisniZahtjev kopijaLaptopa = laptop;
cout << kopijaLaptopa << crt;

Servis fitServis("FIT Servis");

/*
DodajZahtjev dodaje zahtjev u servis. Nije dozvoljeno dodati dva
zahtjeva
sa istim rednim brojem ili istom oznakom. U slucaju duplikata
metoda
baca izuzetak.
*/
fitServis.DodajZahtjev(laptop);
fitServis.DodajZahtjev(telefon);
try {
    fitServis.DodajZahtjev(telefon);
}
catch (exception& e) {
    cout << "Exception: " << e.what() << crt;
}

/*
PronadjiZahtjev vraca pokazivac na zahtjev sa prosljedjenom
oznakom.
Ako zahtjev nije pronadjen, metoda vraca nullptr.
*/
string oznakaLaptopa = laptop.GetOznaka();
ServisniZahtjev* pronadjen =
fitServis.PronadjiZahtjev(oznakaLaptopa);
if (pronadjen != nullptr)
    cout << "Pronadjen zahtjev: " << pronadjen->GetOznaka() <<
crt;
if (fitServis.PronadjiZahtjev("SRV-999/XX-2026") == nullptr)
    cout << "Zahtjev nije pronadjen" << crt;

/*
EvidentirajStatus pronalazi zahtjev i pokusava dodati status
koristeci
```

pravila metode DodajStatus.

Racun se salje u zasebnom threadu isključivo nakon uspješnog evidentiranja

statusa ZAVRSEN. Za ostale statuse se ne salju obavijesti. Ako zahtjev ne postoji ili status nije dodan, metoda vraća false.

Primjer sadržaja racuna:

```
-----
To: amina@fit.ba
From: racuni@servis.ba
Subject: Servisni zahtjev završen - racun
```

Postovani Amina Buric,

Servisni zahtjev SRV-042/AB-2026 za uređaj Laptop je završen.  
Ukupan iznos: 140.50 KM

Hvala na povjerenju.

```
-----
*/
if (fitServis.EvidentirajStatus(oznakaLaptopa, ZAVRSEN, završen))
    cout << "Zahtjev završen i racun poslan" << crt;
if (!fitServis.EvidentirajStatus(
    oznakaLaptopa, ZAVRSEN, DatumVrijeme(9, 7, 2026, 12, 0)))
    cout << "Ponavljanje završnog statusa nije dozvoljeno" << crt;
if (!fitServis.EvidentirajStatus(
    "NEPOSTOJECI", DIJAGNOSTIKA, dijagnostika))
    cout << "Status nije evidentiran za nepostojeci zahtjev" <<
crt;

/*
IzdvojiNezavrsene vraća vector pokazivaca na sve zahtjeve čiji
trenutni
status nije ZAVRSEN.
*/
vector<ServisniZahtjev*> nezavršeni =
fitServis.IzdvojiNezavrsene();
for (auto zahtjev : nezavršeni)
    cout << zahtjev->GetOznaka() << " -> "
    << StatusZahtjevaNazivi[(int)zahtjev->GetTrenutniStatus()] <<
crt;

/*
IzracunajPrihod vraća zbir cijena svih intervencija evidentiranih
na
zahtjevima koji imaju status ZAVRSEN. Intervencije na nezavršenim
zahtjevima ne ulaze u prihod.
*/
cout << "Ostvareni prihod: " << fitServis.IzracunajPrihod()
    << " KM" << crt;

Servis kopijaServisa = fitServis;
cout << kopijaServisa.GetNaziv() << " ima "
    << kopijaServisa.GetZahtjevi().size() << " zahtjeva" << crt;
```

```
cin.get();  
return 0;
```

```
}
```